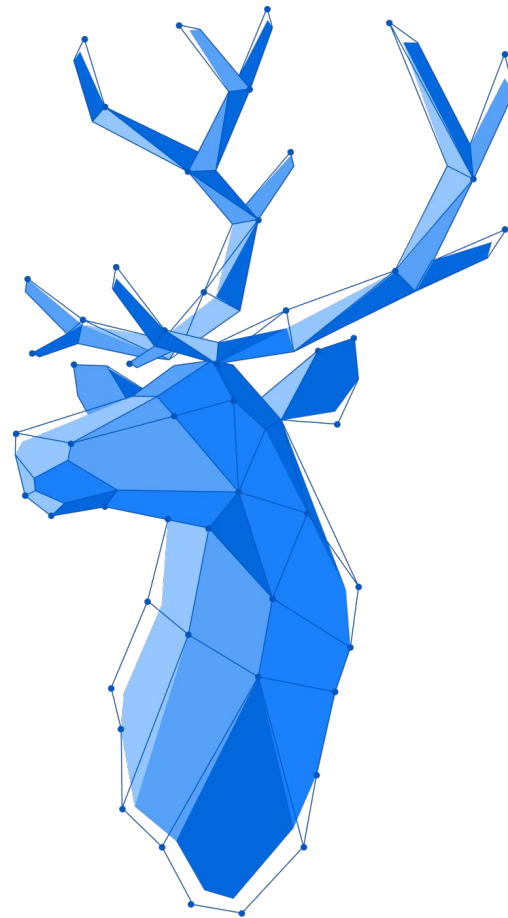




Layer Isolation

Von Gerald Koch



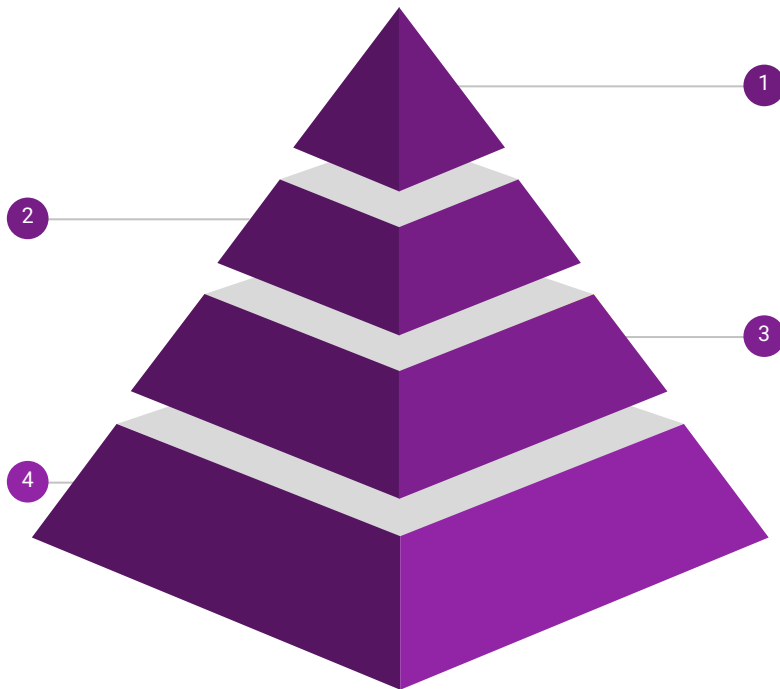
Schichten in Architektur

Presenter

- Benutzt Services um daten zu bekommen
- Dirigiert die View
- Kennt die nötigen Services
- Spricht die View über ein Interface an

Persistenz

- Schicht zur Datenbeschaffung und -Speicherung
- Kann Datenbank, JSON, XML oder andere Formate anbinden
- Sollten ein Interface implementieren, um leicht ausgetauscht zu werden



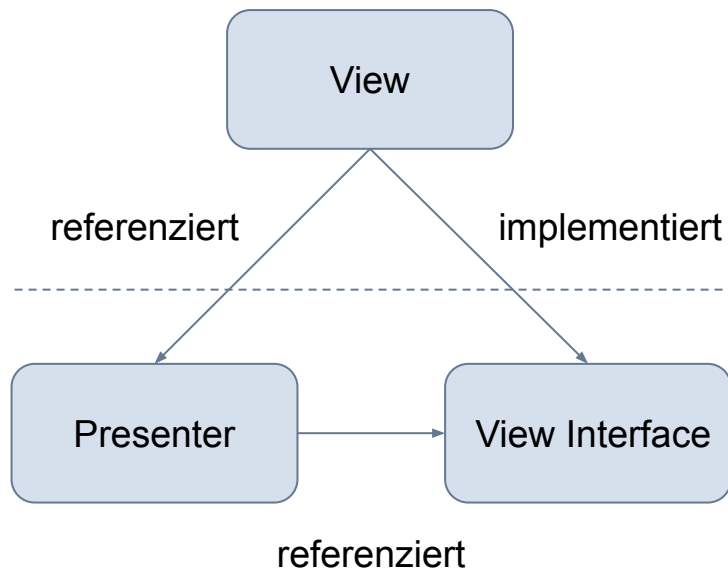
View

- Zeigt an was der Presenter anordnet
- Kennt nur seinen Presenter

Service

- Arbeitet Daten der Persistenz auf und stellt sie den Presentern zur Verfügung
- Kennt die nötigen Persistenzklassen (Angebunden über Interfaces)

Das MVP Pattern



View:

- Implementiert View Interface
- Besitzt UI-Framework-Klassen
- Referenziert Presenter
- Ruft Presenter bei Input-Events auf
- Folgt den Anweisungen des Presenter

View Interface:

- Besitzt Presenter-relevante Methoden
- Wird vom Presenter referenziert
- Keine UI-Framework-Klassen

Presenter:

- Kontrolliert die UI
- Keine UI-Framework-Klassen
- Referenziert das View Interface

Das MVP Pattern – Beispiel Teil 1

View Interface

```
public interface ViewInterface {  
    void showTableData(List<SomeObject> tableData);  
}
```

Presenter

```
@Scope(value =  
        ConfigurableBeanFactory.SCOPE_PROTOTYPE)  
@Component  
public class Presenter {  
  
    private ViewInterface view;  
    private final SomeService service;  
  
    public Presenter (SomeService service) {  
        this.service = service;  
    }  
}
```

```
public void init(ViewInterface view) {  
    this.view = view;  
}  
  
public void buttonClicked() {  
    view.showTableData(service.getSomeBeans());  
}  
}
```

Das MVP Pattern – Beispiel Teil 2

View (Vaadin)

```
@UIScope
@Route("")
public class View extends FlexLayout
    implements ViewInterface {

    private final Presenter presenter;
    private final Grid<SomeObject> grid
        = new Grid<>(SomeObject.class);

    //inject the presenter (Spring)
    public View (Presenter presenter) {
        this.presenter = presenter;
        Button button = new Button("Click",
            e -> presenter.buttonClicked());
        add(button, grid);
    }
}
```

```
@Override
public void onAttach(AttachEvent attachEvent) {
    if(attachEvent.isInitialAttach()) {
        presenter.init(this);
    }
}

@Override
public void showTableData(
    List<SomeObject> tableData) {
    grid.setItems(tableData);
}
}
```

Softwareschichten als Module

